

Entrenamiento de un tutor de programación con Fine-Tuning y LoRA

Creación del dataset

El proceso inició con la creación de un dataset en formato *.jsonl* mediante el script *crear_dataset*. Este dataset contiene preguntas y respuestas de programación que abarcan desde conceptos básicos hasta temas más complejos como inteligencia artificial. Su propósito es proporcionar al modelo un contexto pedagógico amplio y coherente.

Se generaron **143 ejemplos**, todos siguiendo un formato compatible con modelos *instruction-based*.

```
{"instruction": "¿Qué es la regularización?", "response": "Regularización agrega una penalización al error del modelo para evitar overfitting. Tipos comunes: L1 (Lasso) que puede eliminar características, L2 (Ridge) que reduce magnitudes de coeficientes. Controla complejidad del modelo. En redes neuronales, dropout desactiva aleatoriamente neuronas durante entrenamiento. Ayuda a generalizar mejor a datos nuevos."}
```

Fine-tuning del modelo base

El archivo *.jsonl* fue cargado para realizar el fine-tuning sobre el modelo preentrenado *mistralai/Mistral-7B-Instruct-v0.2*. Este modelo cuenta con aproximadamente **7 mil millones de parámetros** y está diseñado para seguir instrucciones en lenguaje natural.

Durante esta etapa se configuró el tokenizer correspondiente, estableciendo el token de padding como el token de fin de secuencia para garantizar compatibilidad durante el entrenamiento.

```
# ===== CARGAR MODELO =====  
model_name = "mistralai/Mistral-7B-Instruct-v0.2"  
tokenizer = AutoTokenizer.from_pretrained(model_name)  
tokenizer.pad_token = tokenizer.eos_token  
tokenizer.padding_side = "right"
```

Adaptación mediante LoRA

Para reducir el costo computacional del entrenamiento, se utilizó LoRA (Low-Rank Adaptation). Esta técnica permite ajustar el comportamiento del modelo entrenando únicamente adaptadores de bajo rango en capas específicas de atención, sin modificar todos los parámetros del modelo base.

El uso de LoRA permite que el modelo adopte el estilo explicativo del dataset, logrando respuestas más claras y consistentes en su rol de tutor.

```
# ===== CONFIGURAR LORA =====  
lora_config = LoraConfig(  
    r=16,  
    lora_alpha=32,  
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],  
    lora_dropout=0.05,  
    bias="none",  
    task_type="CAUSAL_LM"  
)  
  
model = get_peft_model(model, lora_config)  
model.print_trainable_parameters()
```

Uso de Google Colab para entrenamiento e inferencia

El entrenamiento y las pruebas se realizaron en Google Colab, aprovechando la GPU y la memoria RAM proporcionadas por la plataforma. Esto permitió reducir significativamente los tiempos de entrenamiento y obtener respuestas más rápidas durante la fase de prueba.

Carga del modelo entrenado

Una vez finalizado el entrenamiento, se generó la carpeta *lora-tutor-final* con los adaptadores entrenados. Para la inferencia, el modelo base fue cargado nuevamente y se le aplicaron los adaptadores LoRA. El modelo se configuró con cuantización a 8 bits para optimizar el uso de memoria.

```
# ===== CARGAR MODELO (solo una vez) =====
print("🔧 Cargando modelo...")
print("⌚ Esto tomará 1-2 minutos...\n")

tokenizer = AutoTokenizer.from_pretrained(RUTA_ADAPTADORES)
tokenizer.pad_token = tokenizer.eos_token

base_model = AutoModelForCausalLM.from_pretrained(
    MODELO_BASE,
    device_map="auto",
    torch_dtype=torch.float16,
    low_cpu_mem_usage=True,
    offload_folder="offload",
    load_in_8bit=True
)

model = PeftModel.from_pretrained(base_model, RUTA_ADAPTADORES)
model.eval()

print("✅ Modelo listo!\n")
print("⚠️ AVISO: Cada respuesta tardará ~1-3 minutos en CPU")
print("=" * 60)
```

Ejemplos de preguntas y respuestas

```
{"instruction": "¿Qué es TOKEN WEB JSON?", "response": "Token Web JSON (JWT) es un estándar para representar información de usuario en un token. Contiene cabecera, carga útil y firma. Se usa en autenticación y autorización. Puede firmarse con algoritmos simétricos (HS256) o asimétricos (RS256). Es vulnerable a ataques si no se maneja correctamente. Se valida verificando la firma y la expiración."}
```

```
{"instruction": "¿Qué es un ciclo?", "response": "Un ciclo es una estructura de control que permite ejecutar un bloque de código repetidamente. Los tipos más comunes son for y while. Se utilizan para iterar sobre colecciones o repetir instrucciones hasta que se cumpla una condición. Es importante controlar su salida para evitar bucles infinitos."}
```

```
{"instruction": "¿Qué es un hashmap?", "response": "Un hashmap es una estructura de datos que almacena pares clave-valor y permite acceso rápido a los datos mediante una función hash. Es útil cuando se requiere búsqueda eficiente. En Python, el tipo dict es una implementación de hashmap."}
```

Resultado final

El modelo entrenado es capaz de responder preguntas de programación con explicaciones claras, ejemplos y contexto práctico. El fine-tuning con LoRA permitió especializar un modelo grande de forma eficiente, obteniendo un tutor de programación funcional que puede ejecutarse tanto en la nube como en entornos locales.